

CE2007

Microprocessor-based System Design

ARM Cortex-M Architecture

Topics Covered

- Introduction to Cortex-M Processor Family
- Cortex-M Architecture
 - Programming Model
 - Startup Sequence
 - Memory Map
 - Address Decoding

Reference: Joseph Yiu, "The Definitive Guide to the ARM Cortex M0 (M3)"

ARM Processor Families (Cortex)

- Cortex-A (based on the ARMv7-A architecture)
 - Application processors that are required to support complex applications like smart phones and PDAs
- Cortex-R (based on the ARMv7-R architecture)
 - Real-time, high performance processors to support highly demanding applications e.g. hard-disk controller and automotive control systems
- Cortex-M
 - Microcontroller processors for industrial control e.g. generic microcontrollers or cost sensitive embedded systems e.g. low cost consumer products
 - Examples: Cortex-M3, Cortex-M0/M0+, Cortex-M1, Cortex-M4/M4F

- Cortex-M3
 - For embedded applications requiring fast interrupt response such as microcontrollers for automotive and industrial control systems
- Cortex-M0 / M0+
 - Smallest ARM processor (optimized for low area and power)
- Cortex-M1
 - For embedded applications that require a small processor to be integrated into an FPGA
- Cortex-M4
 - Enhanced with DSP instructions to address digital signal control markets
 - Cortex-M4F includes floating-point arithmetic functionality

Cortex-M Instruction Set

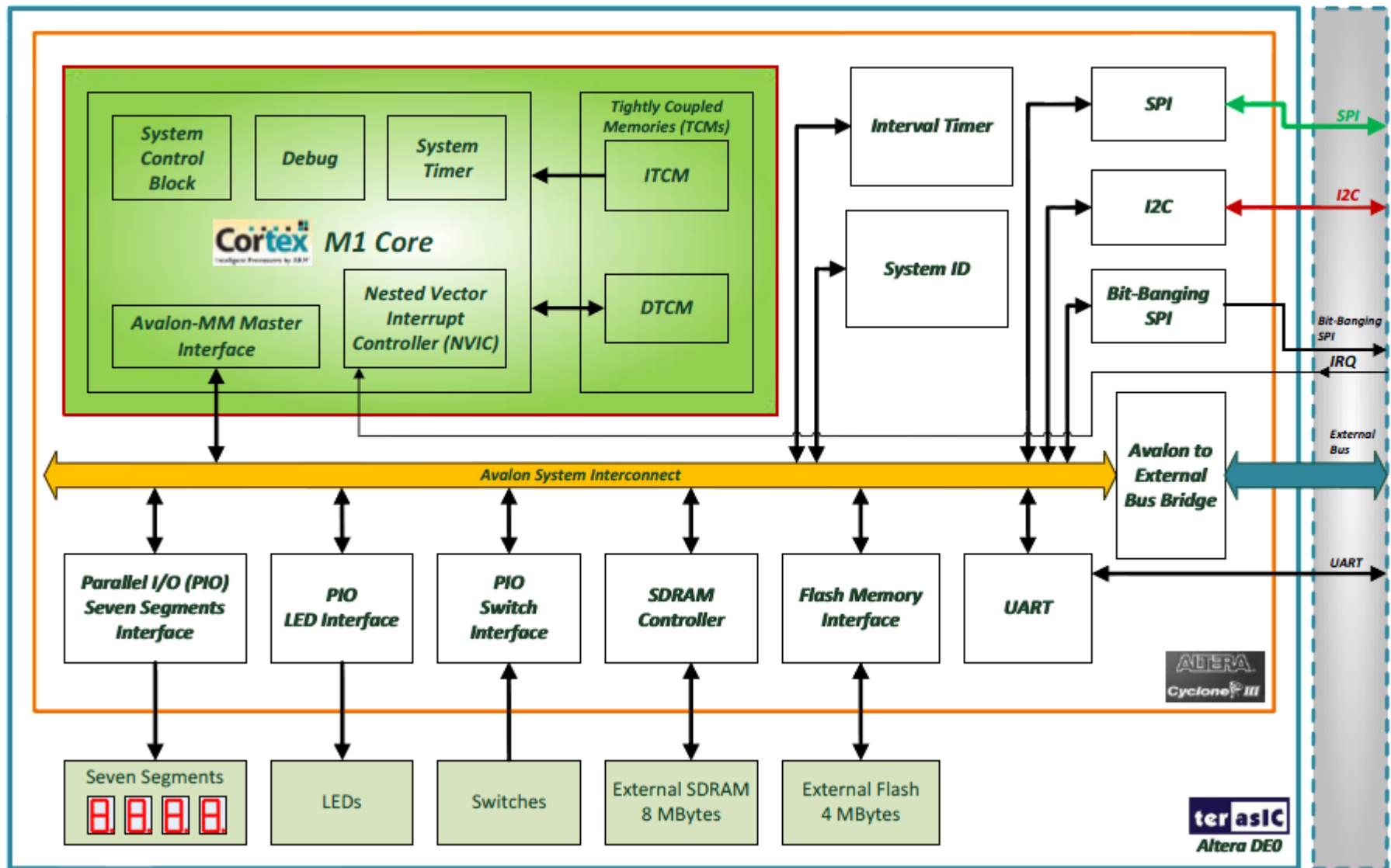
Cortex-M0/M1

Cortex-M3

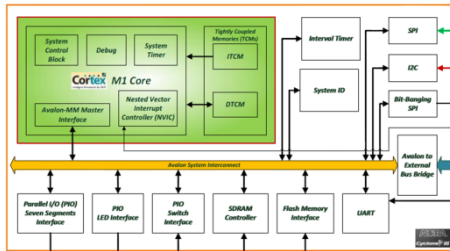
Cortex-M4 FPU

Cortex-M
Instruction Set
Source:
http://www.embedded-world.eu/fileadmin/user_upload/pdf/arm_entwicklerkonferenz_2011/Session_1/01%20-%20Cortex-M%20architecture.pdf

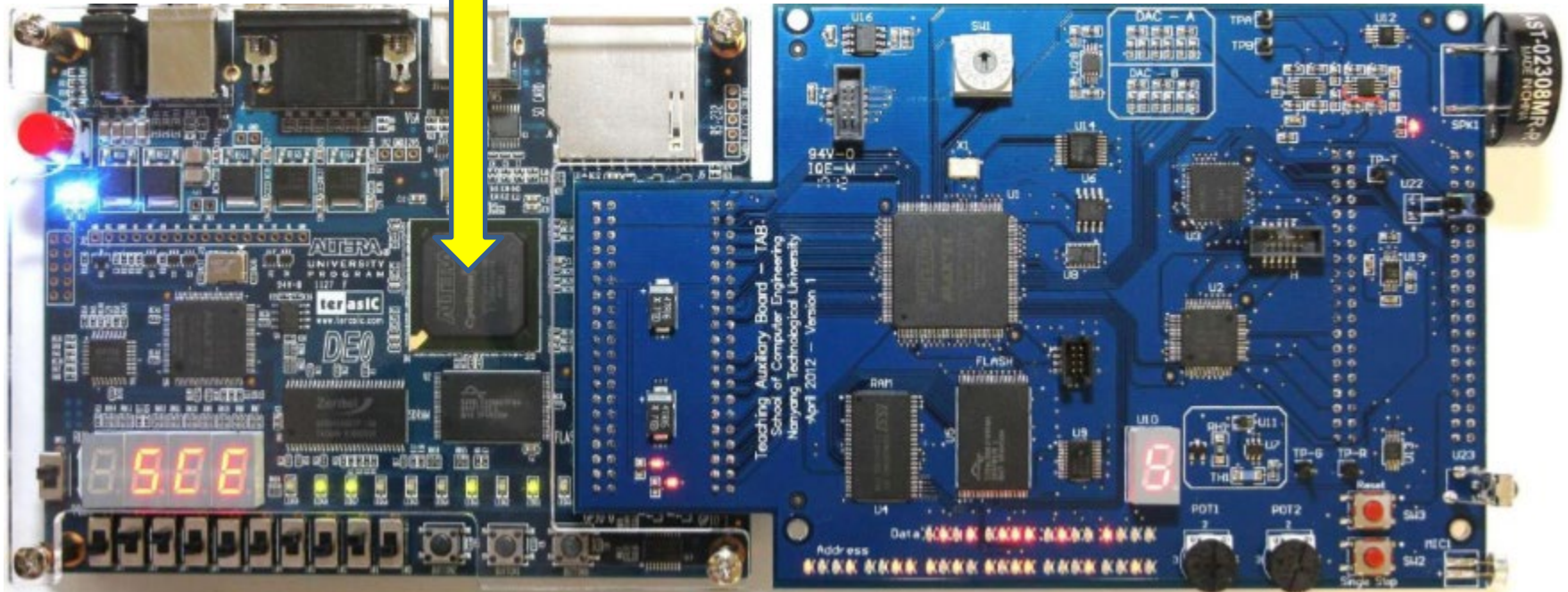
Cortex-M1 on the DE0 Board



CE2007 Cortex-M1 Development System



**Cortex-M1
implemented on
FPGA**

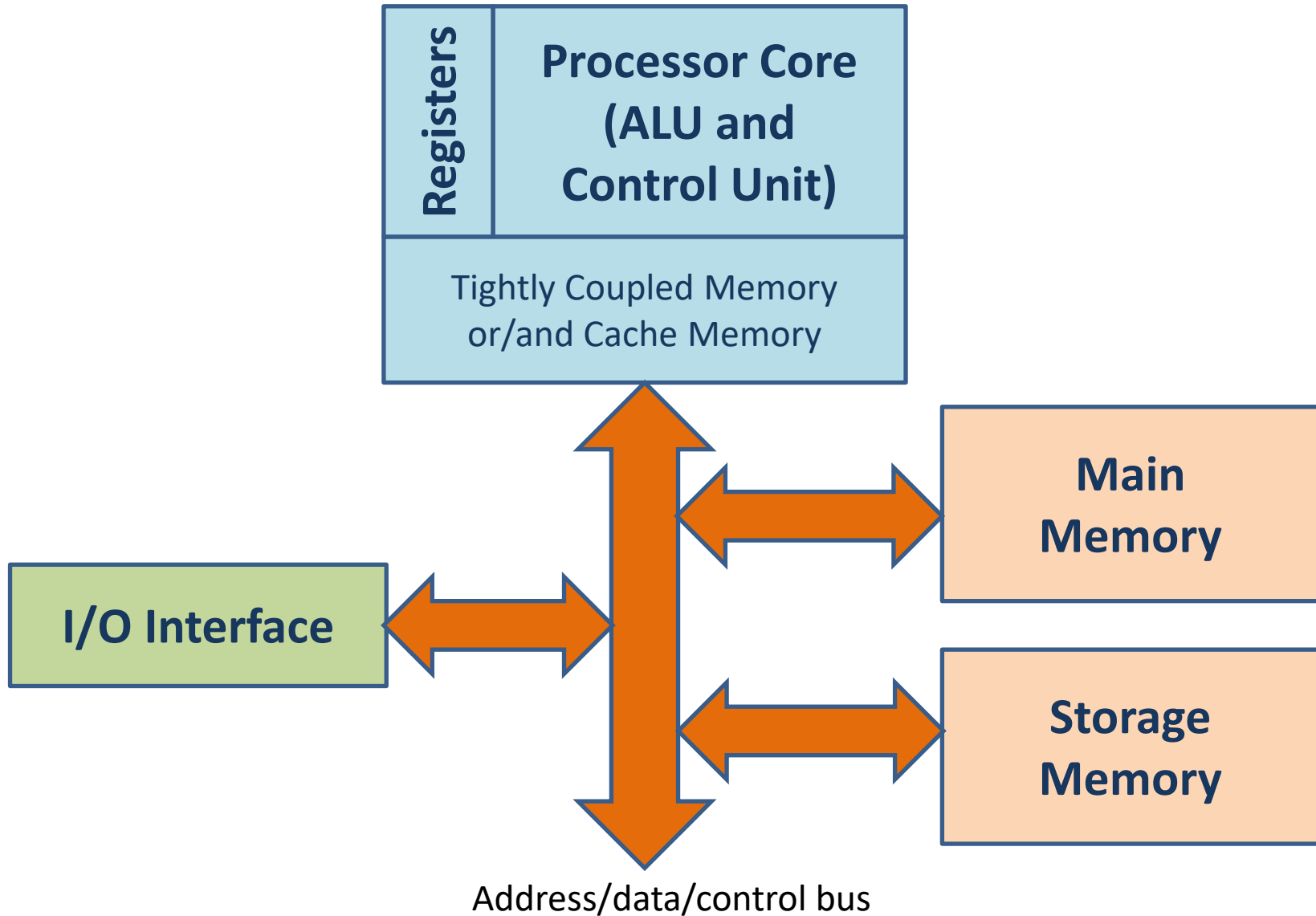


**DE0 FPGA
Development Kit**

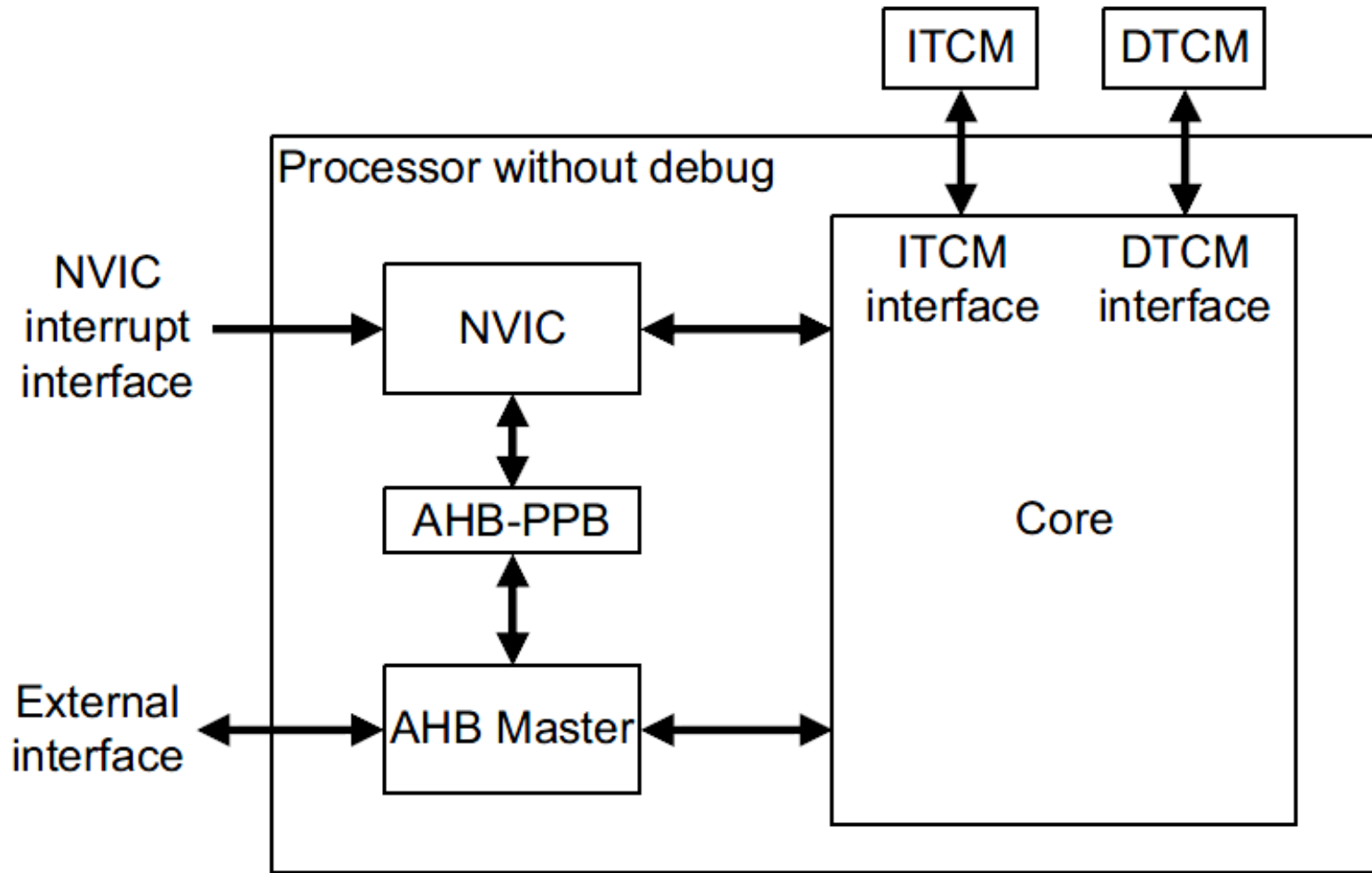
**Teaching Auxiliary
Board (TAB)**

Cortex-M Architecture

Basic Computer System



Cortex-M1 Processor Block Diagram



Source: Cortex-M1 Technical Reference Manual

Cortex-M1 Core Components

- Core memory interfaces comprises of:
 - *Instruction Tightly-Coupled Memory* (ITCM) interface to access ITCM that stores **code**
 - *Data Tightly-Couple Memory* (DTCM) interface to access DTCM that stores **data**
- NVIC (Nested Vectored Interrupt Controller)
 - Tightly coupled to the processor core to facilitate low latency exception processing
- Bus master
 - To access other memory and peripherals
- AHB-PPB (AHB Private Peripheral Bus)
 - To access NVIC via the bus master

Question 1

Is the Cortex-M1 processor a Harvard or Von-Neumann architecture?

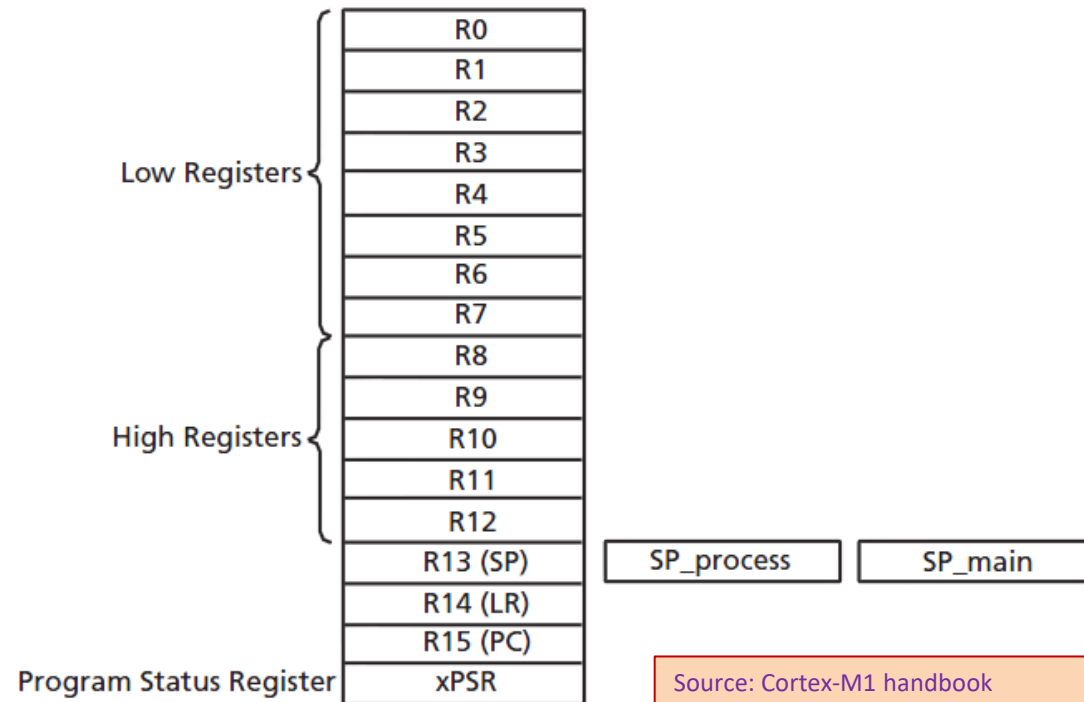
- A. Harvard
- B. Von-Neumann

Cortex-M

Programming Model

Cortex-M Registers

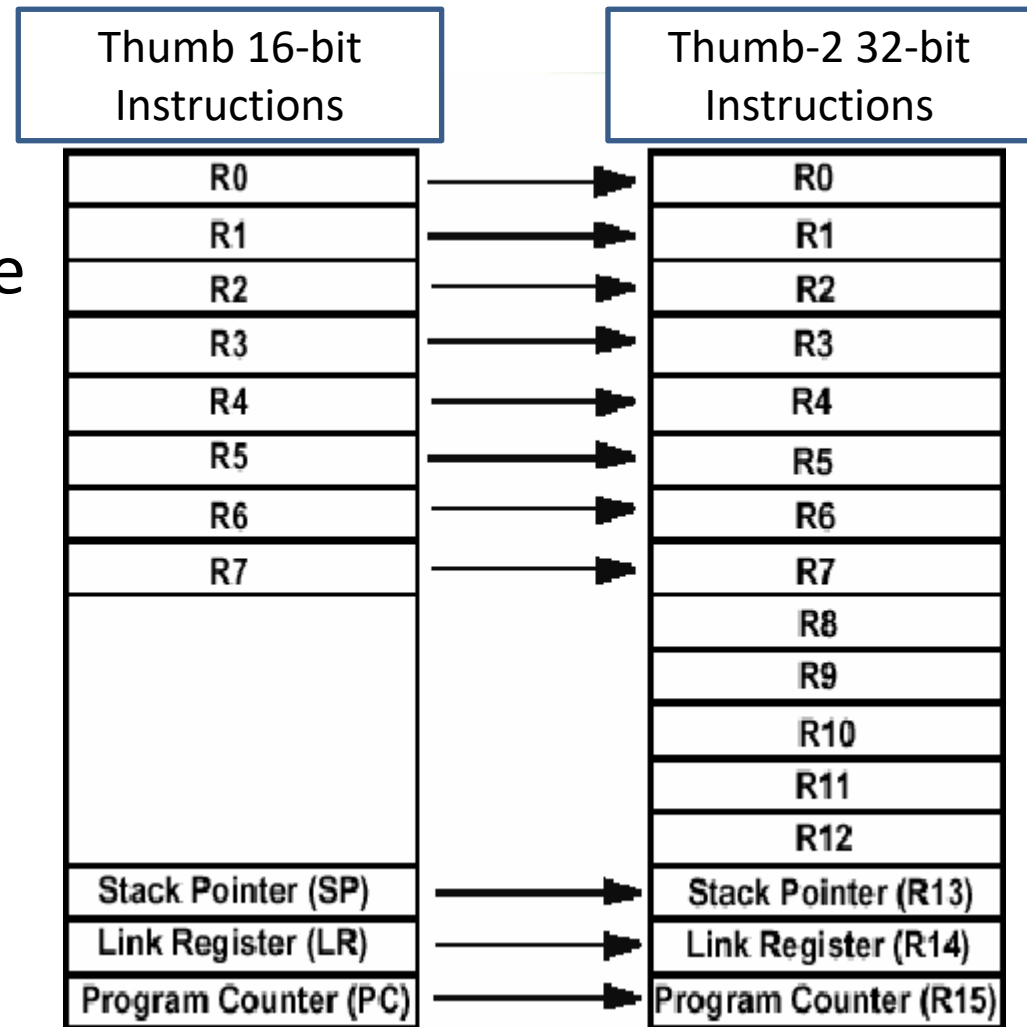
- Cortex-M processor has the following 32-bit registers
 - 13 General Purpose registers: R0 to R12
 - 2 (banked) Stack Pointer (MSP and PSP) registers: R13
 - 1 Link Register (LR): R14
 - 1 Program Counter (PC): R15
 - 1 Program Status Register (xPSR)



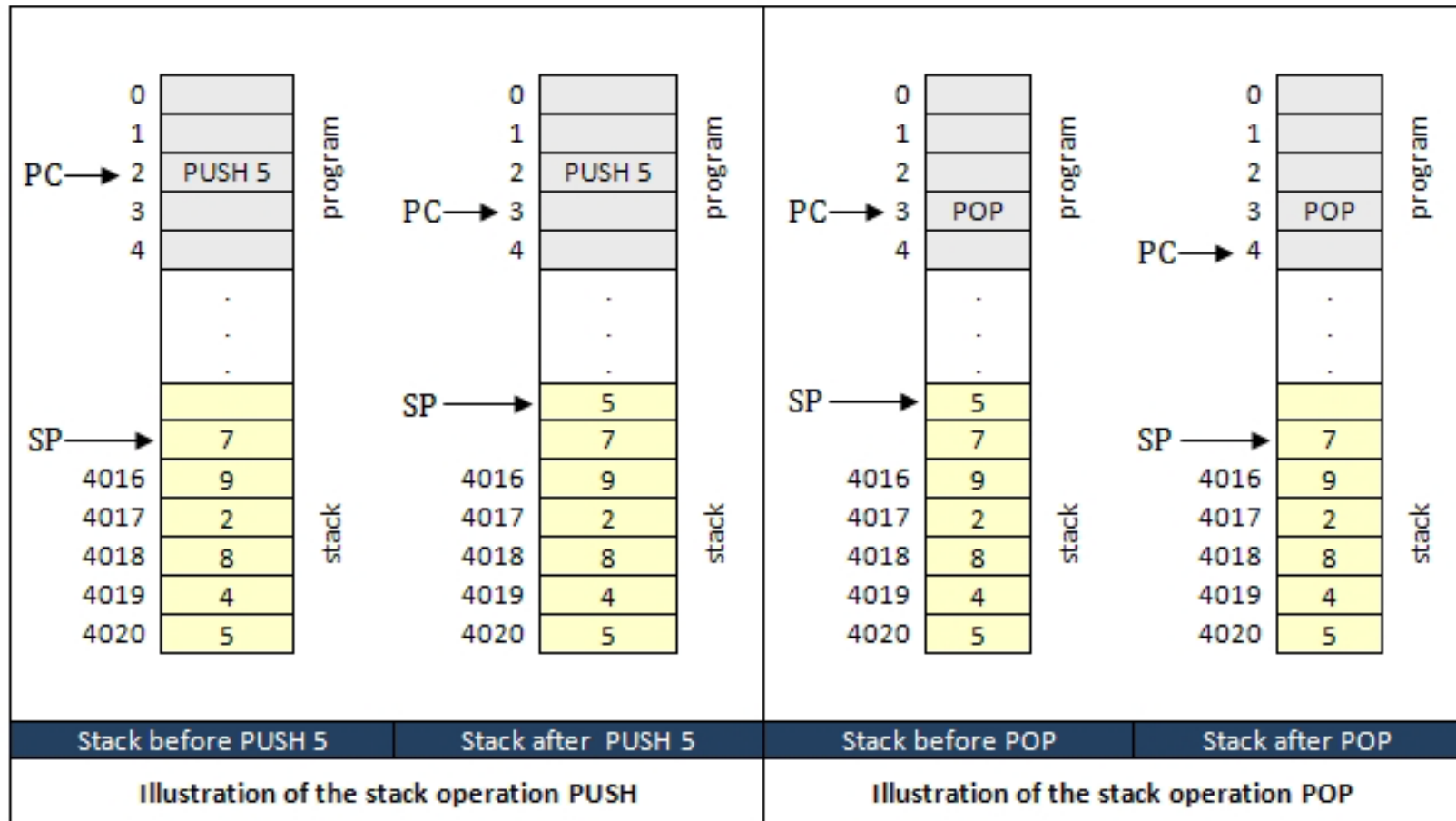
(There are a few more registers which we will skip for the time being)

R0 to R12

- General purpose registers that can contain **data** or **addresses**
- Low registers (R0 – R7)** are generally not restricted in their usage (e.g. when used in assembly language programming)
- High registers (R8 – R12)** are not accessible by all 16-bit Thumb instructions



Revision – Stack Pointer and Program Counter



Source: <http://phaariz.wordpress.com/2009/10/02/stack-operations-push-and-pop/>

Question 2

What is the stack overflow problem? Think of a programming technique that you can use to detect stack overflow.

(To be discussed in lectures)

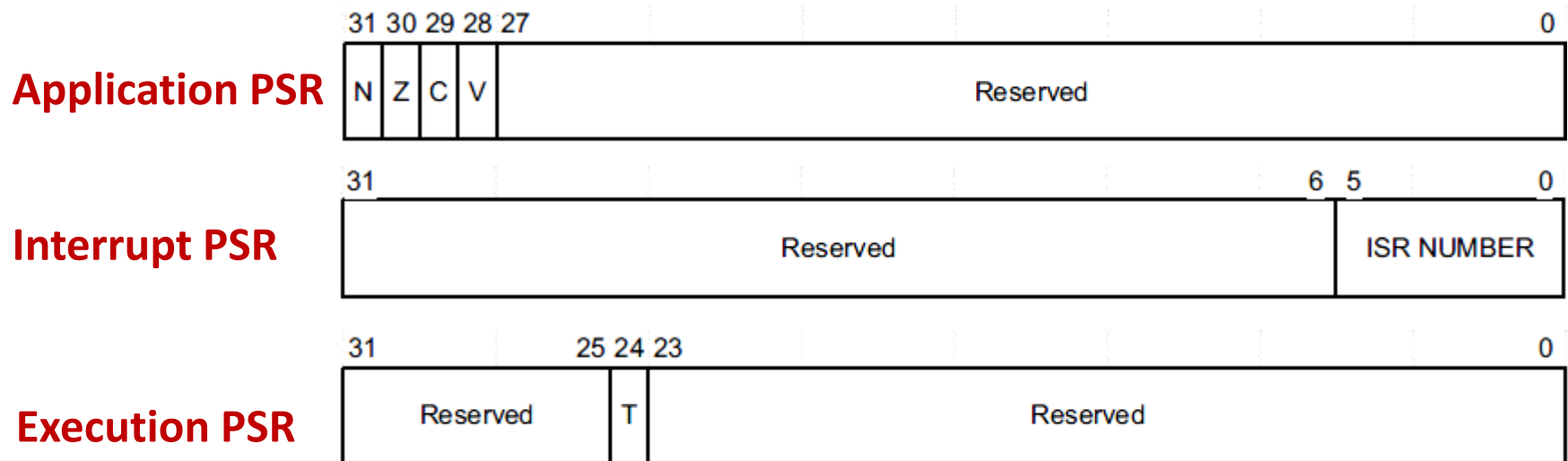
R13, Stack Pointer (SP)

- Store address (pointer) to the **top of the stack** that is used to store 32-bit data (stack is **descending** in Cortex-M implementation)
- There are two stack pointers: **Main Stack Pointer (MSP)** and **Process Stack Pointer (PSP)**
 - But only one is visible at R13 at only one time
 - Example: MSP can be used when processor is servicing an interrupt while PSP is used in normal operation
 - For simple embedded applications, only MSP is usually used (default SP upon Reset)

- **R14: Subroutine Link Register (LR)**
 - Stores the return address when an exception (i.e. interrupt) occurs
 - Address in the Program Counter (PC) is copied to LR when executing the Branch and Link (BL) instruction (equivalent to a JMP or CALL operation)
- **R15: Program Counter (PC)**
 - Contains the address of the next instruction to be fetched from memory

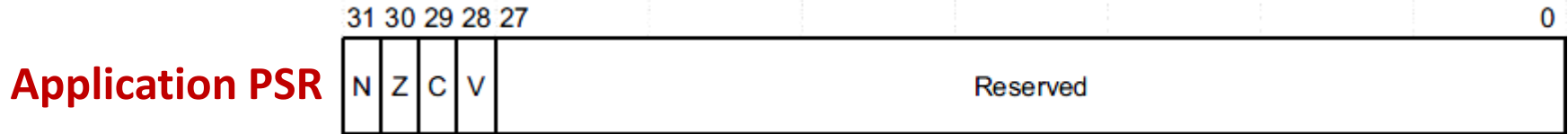
Program Status Register

- Combined **Program Status Register** provides information about program execution and ALU flags
- It consists of the following three registers:



- On entering an exception, the processor saves the combined information from the three registers as a single register (xPSR) on the stack

Application Program Status Register (APSR)



- Contains the condition code flags based on result of previous ALU operations
- Common use of these flags is to control conditional branches

Field	Name	Definition
[31]	N	Negative or less than flag: 1 = result negative 0 = result positive.
[30]	Z	Zero flag: 1 = result of 0 0 = nonzero result.
[29]	C	Carry or borrow flag: 1 = carry true or borrow false 0 = carry false or borrow true.
[28]	V	Overflow flag: 1 = overflow 0 = no overflow.
[27:0]	-	Reserved ^a

Question 3

Think of a scenario where using the Link Register to store the return address of subroutines will not work.

(To be discussed in lectures)

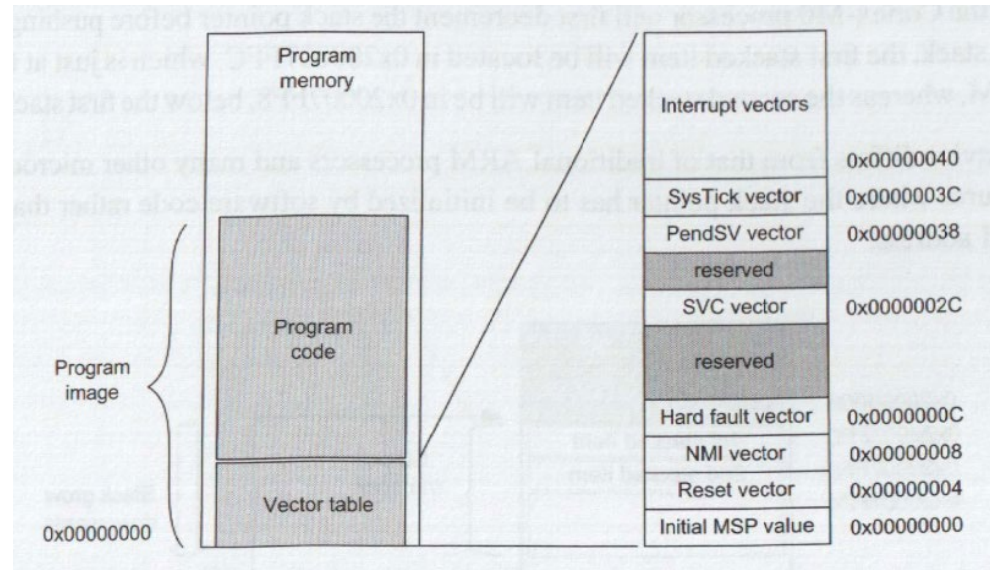
Cortex-M

Startup Sequence

Start-Up Sequence

- The program image for Cortex-M1 is located from address **0x00000000** and it consists of the vector table and program code

- Processor reads first word at location **0x00000000** and stores the value into stack pointer **MSP**

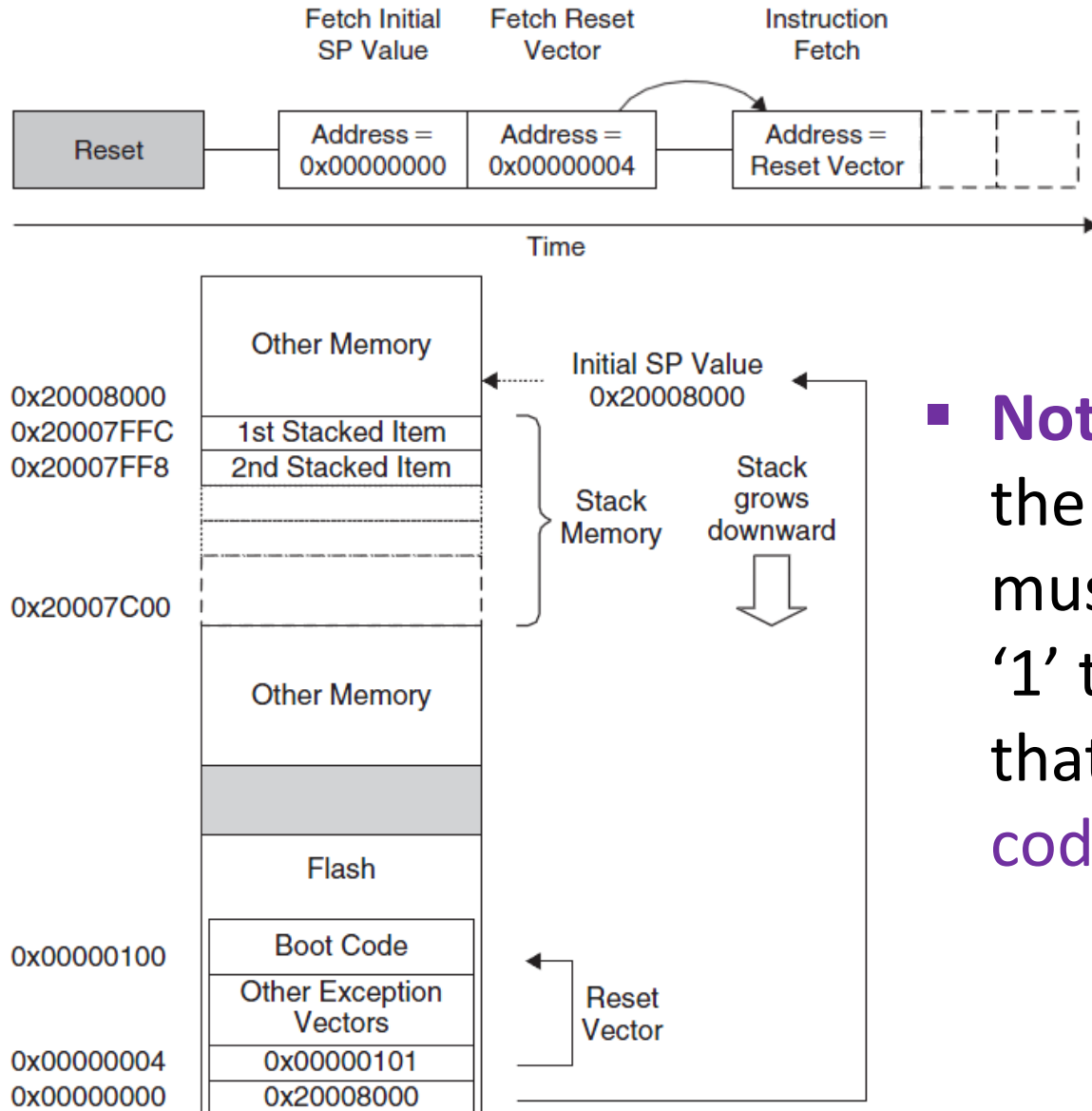


Vector table in program image

Source: The Definitive Guide to the ARM Cortex-M0; 1st Edition; Joseph Yiu; 2011

- Processor then reads the word at location **0x00000004** and copy it to the program counter **PC**
 - This is the **reset vector (address)** which determines the starting of the program execution address

Start-Up Sequence (Cont.)

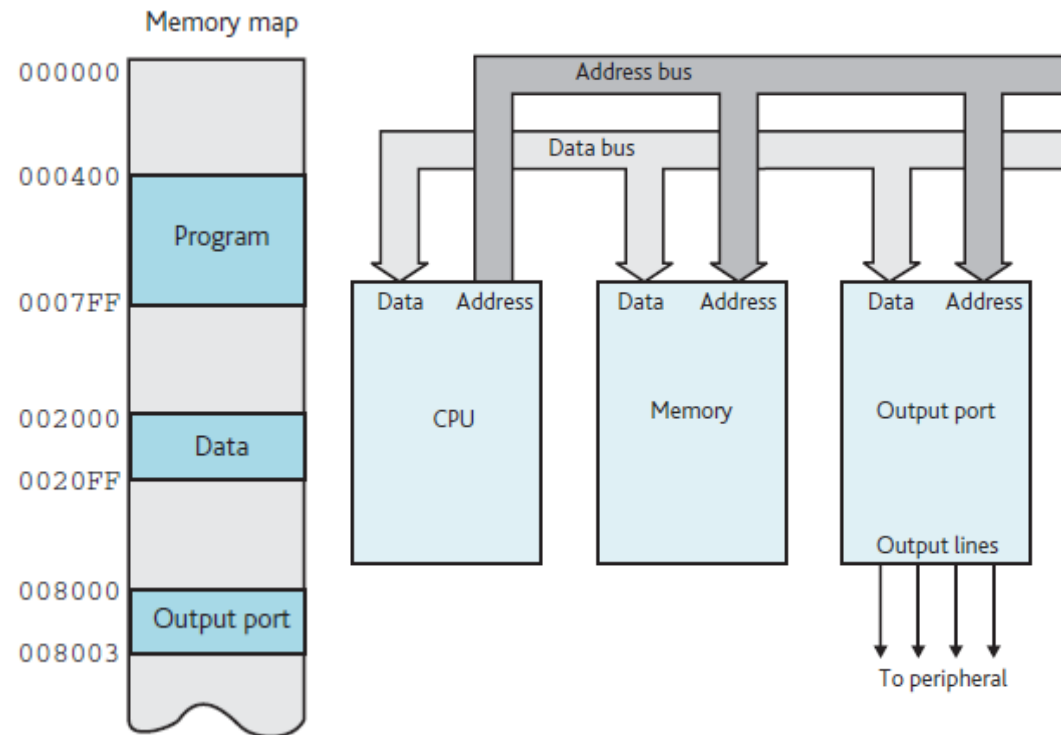


- **Note:** Bit 0 of the reset vector must be set to '1' to indicate that it is **Thumb code**

Cortex-M Memory Map

Memory Map

- Address space refers to the locations where the various memory devices (ROM and RAM) and I/O peripherals can be accessed
- ARM processor families utilize only one address space for storing both code/data as well as accessing I/O devices
 - I/O devices are **memory mapped** to the same address space as code and data memory



Source: Clements, "Principles of Computer Hardware", Oxford, Fourth Edition

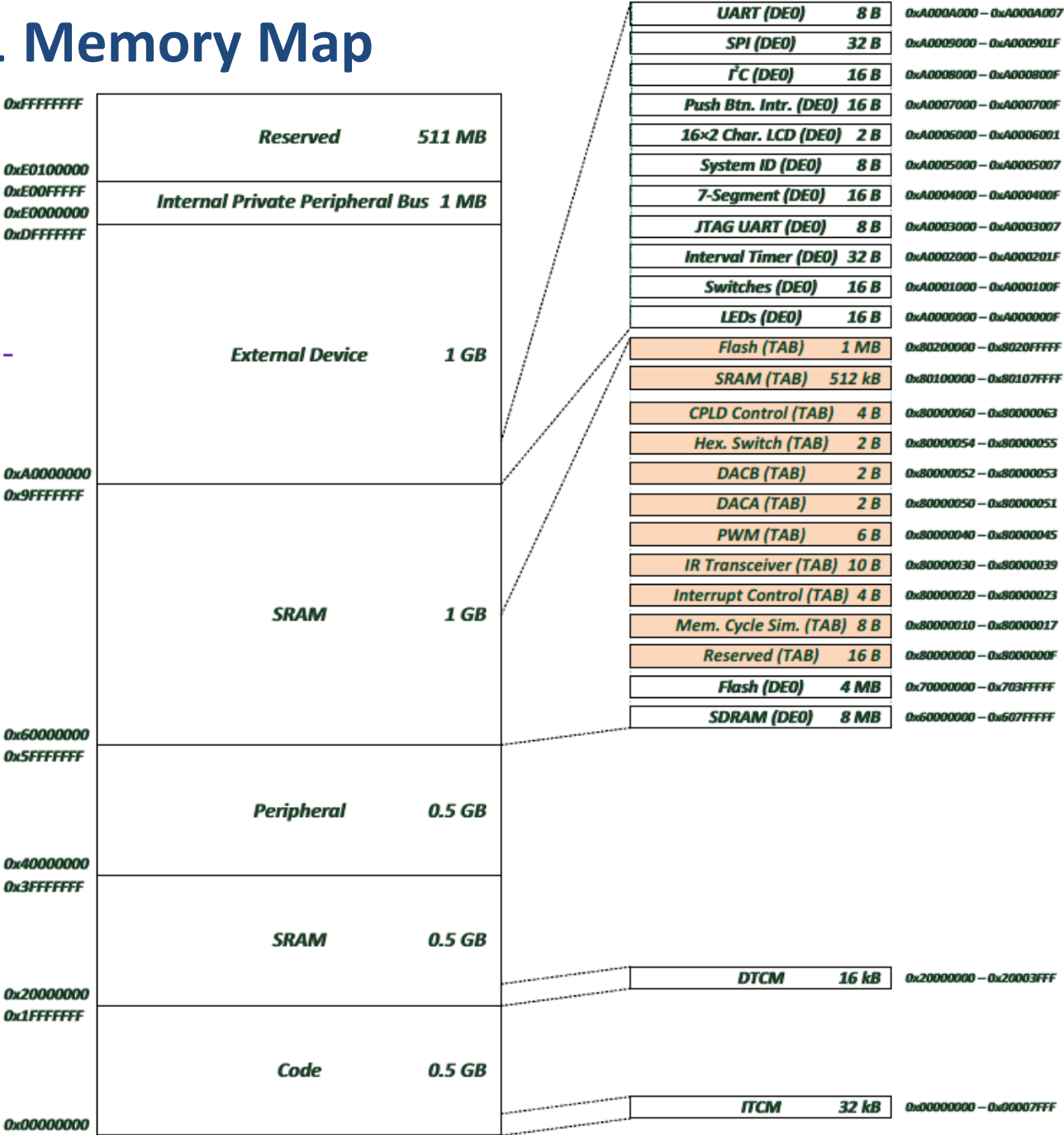
Cortex-M Memory Map

- Different implementations of Cortex-M by different manufacturers
 - Provide different amount of memory and different types of I/O devices and peripherals
 - But similar memory map
 - Address space of the Cortex-M1 processor is partitioned into specific segments
- Examples:
 - Flash/SRAM for code (ITCM) begins at 0x00000000
 - SRAM for data (DTCM) begins at 0x20000000
 - I/O peripherals are mapped to 0x40000000 – 0x5FFFFFFF
 - I/O modules connected to private peripheral bus are mapped to 0xE0000000 – 0xE00FFFFFFF

DE0 Cortex-M1 Memory Map

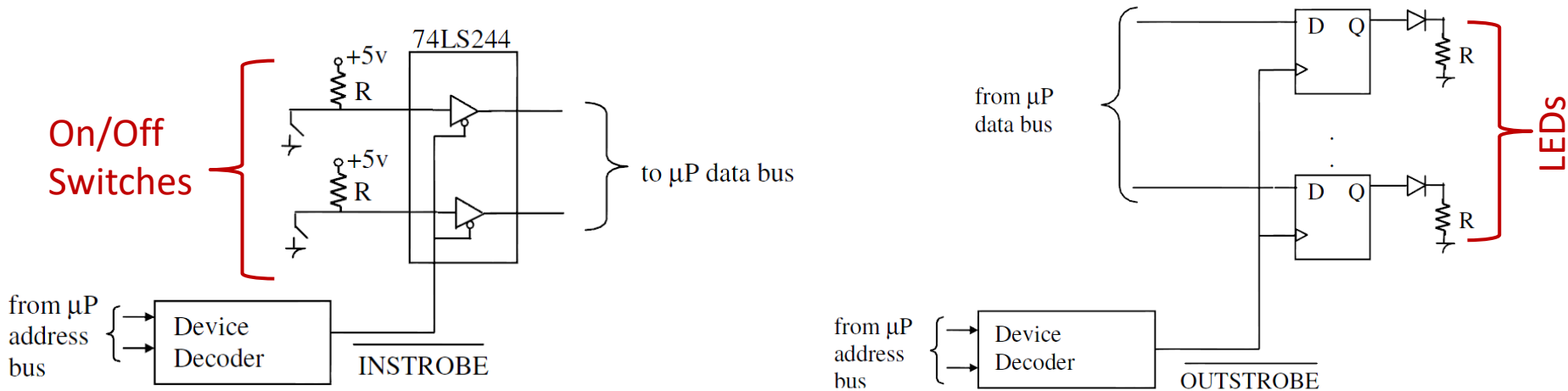
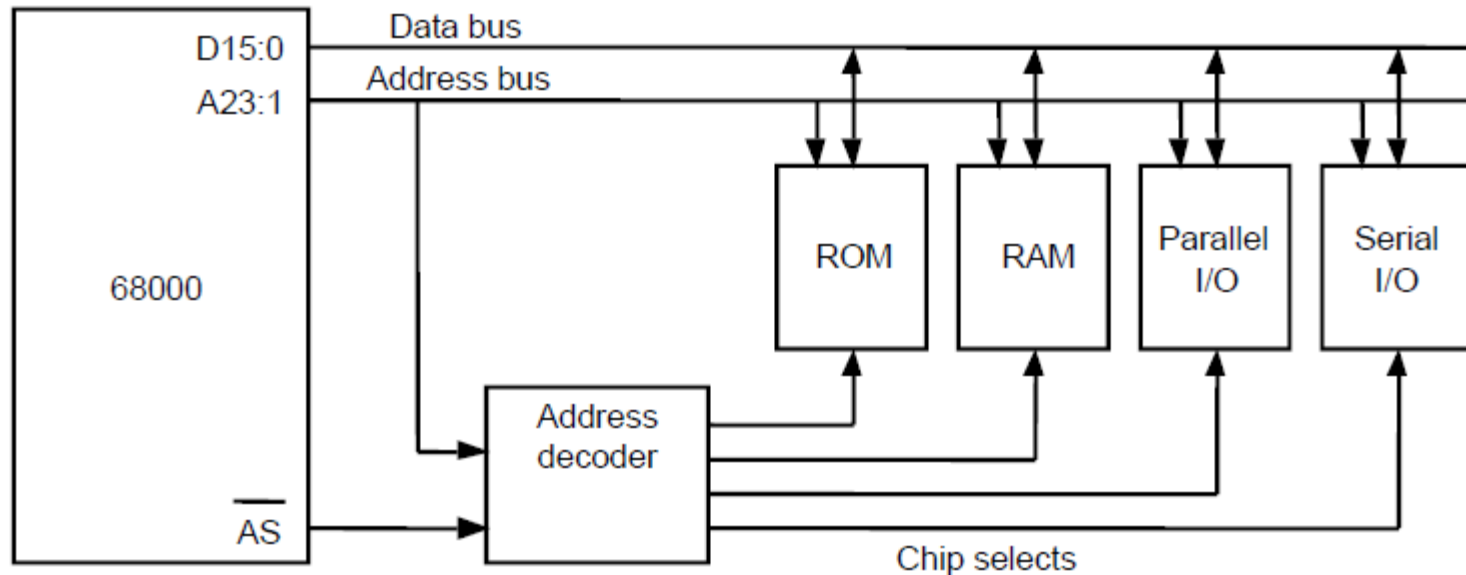
Note that SDRAM and Flash memory on DE0 board, Flash and other components on TAB board are mapped to pre-defined SRAM memory region of Cortex-M1 processor at address range **0x60000000 – 0x9FFFFFFF**

Both ITCM and DTCM utilize on-chip memory residing in the **FPGA**



Hardware Interfacing with Peripheral Devices

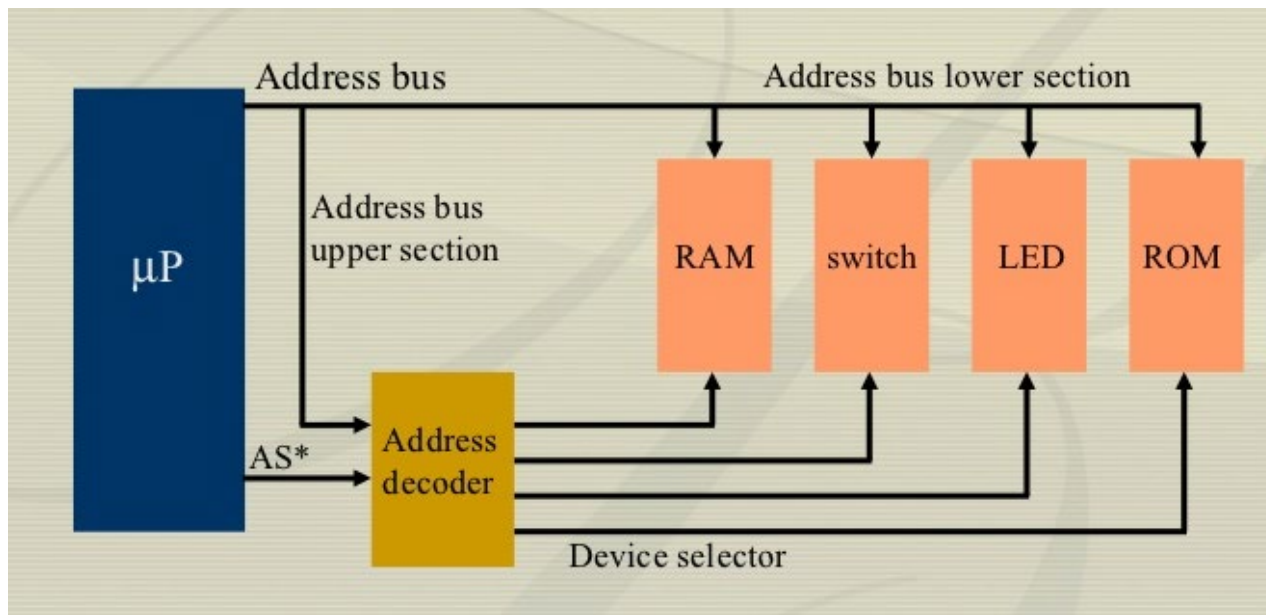
Example:



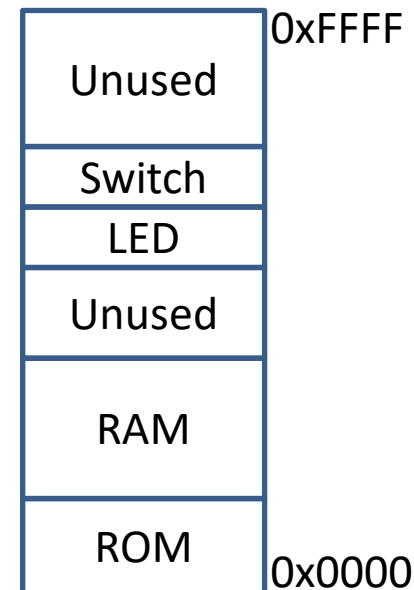
Address Decoders

Why do we Need Address Decoders?

- Microprocessor systems often have memory components that are smaller than the addressable memory space.
- Different portions of memory are mapped to different peripherals: RAM, ROM, Memory-Mapped Peripherals
- For a given valid address, only one component must be accessed.

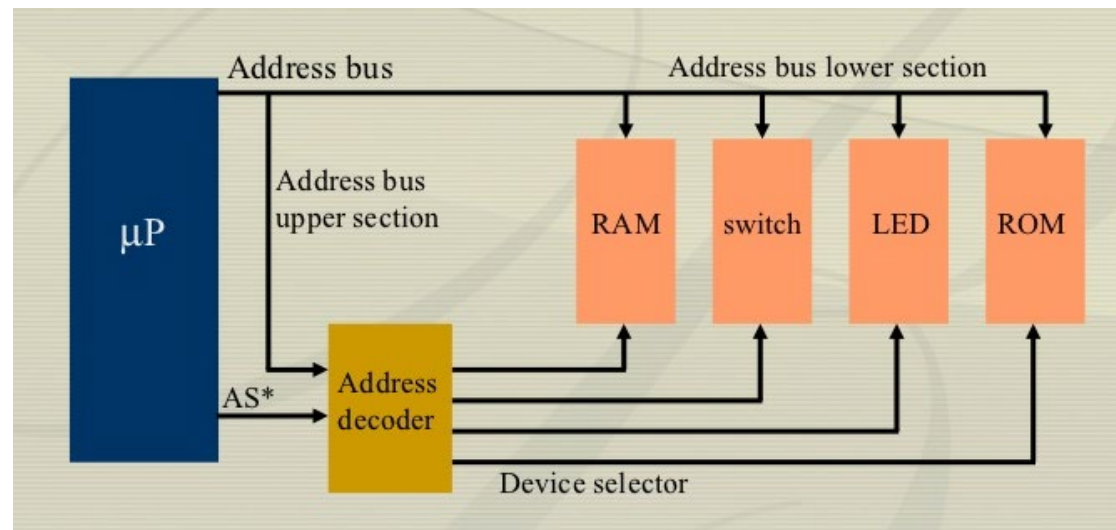
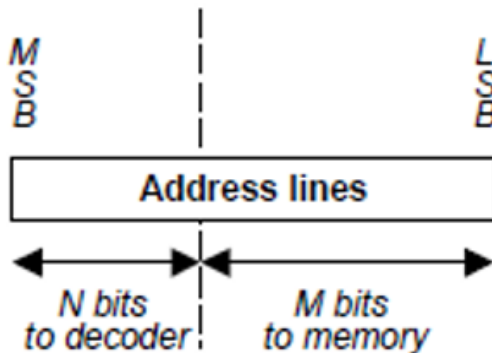


**Address Space
(Memory Map)**



Address Decoding

- Address Decoding is the process of generating Chip Select ($\overline{CS}$) signals from the address bus for each device in the system.
- The address bus lines are split into two sections:
 - The N most significant bits are used to generate the ($\overline{CS}$) signals for the different devices
 - The M least significant signals are passed to the devices as addresses to the different memory cells or internal register.

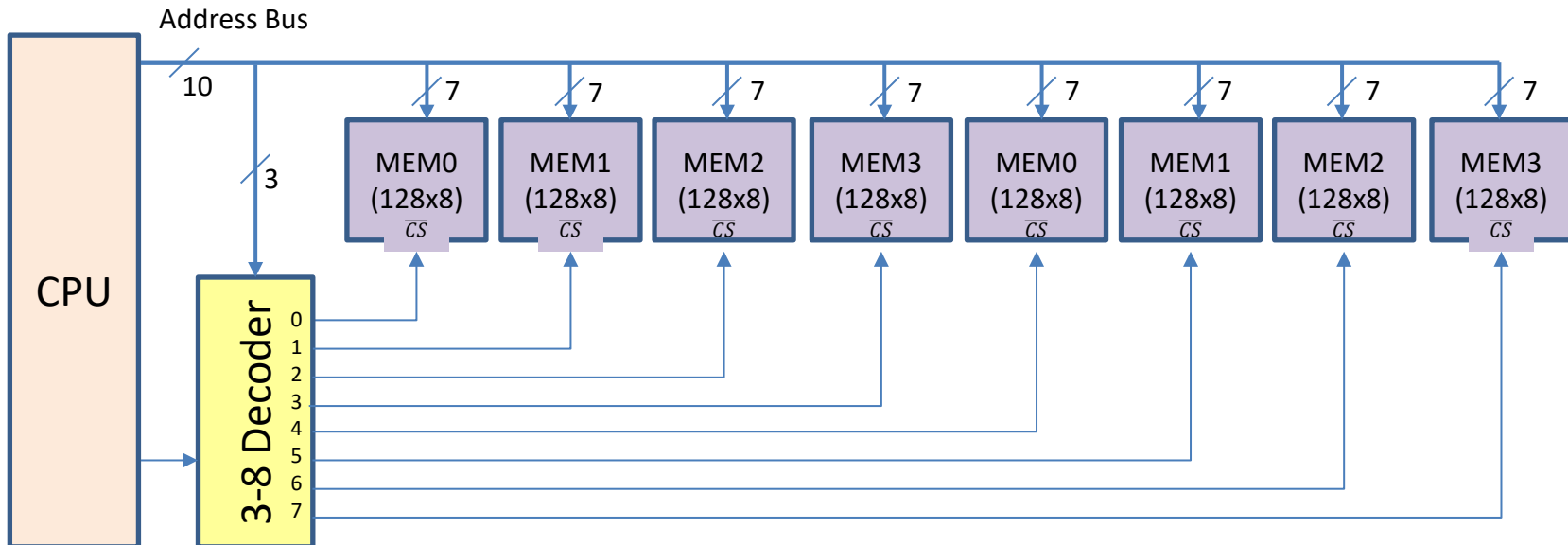


Simple Address Decoding Example

- A microprocessor has 10 address lines. Its addressable space is $2^{10} \Rightarrow 1\text{KB}$ of memory.
- We wish to populate its entire memory space with 128×8 memory chips.
- Solution:
 - We need 8 memory chips $\Rightarrow 8 \times 128 = 1024$
 - Each chip will need 7 address lines to address its internal memory cells.
 - We will need 3 address lines to select each one of the 8 chips.

Memory Map

MEM 7	111 1111 ₂ 000 0000 ₂
MEM 6	111 1111 ₂ 000 0000 ₂
MEM 5	111 1111 ₂ 000 0000 ₂
MEM 4	111 1111 ₂ 000 0000 ₂
MEM 3	111 1111 ₂ 000 0000 ₂
MEM 2	111 1111 ₂ 000 0000 ₂
MEM 1	111 1111 ₂ 000 0000 ₂
MEM 0	111 1111 ₂ 000 0000 ₂

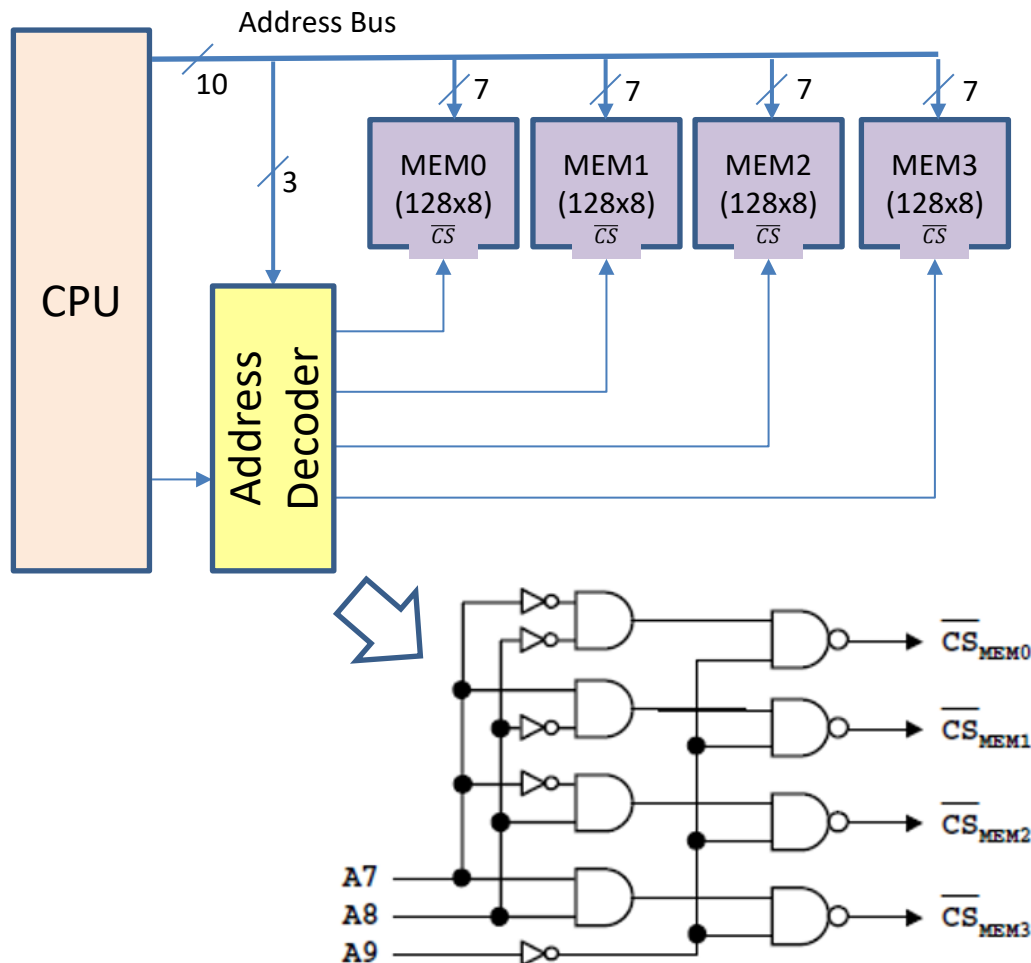


- In the previous example we populated the entire address space. Generally, only a portion of the addressable space is going to be used.
- Two basic addressing decoding strategies:
 - Full Address Decoding
 - All the address lines are used to specify a memory location
 - Each physical memory location is identified by a unique address.
 - Partial Address Decoding
 - Only a subset of address lines are needed to point to the physical memory locations.
 - Each physical memory location is identified by several possible addresses.

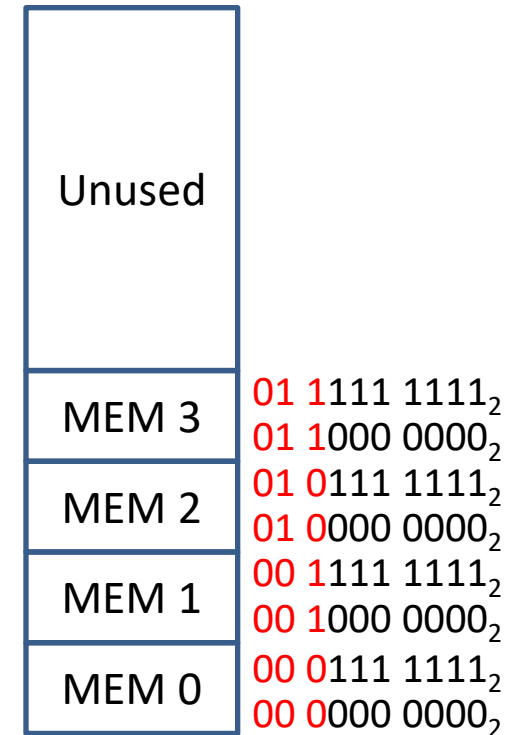
Full Address Decoding

- A microprocessor has 10 address lines
Its addressable space is $2^{10} \Rightarrow$ 1KB of memory.
- Now we only want 512 bytes of memory.

Device	Used for Address Decoding			Used to reference memory cells on each memory IC						
	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0
MEM 0	0	0	0	X	X	X	X	X	X	X
MEM 1	0	0	1	X	X	X	X	X	X	X
MEM 2	0	1	0	X	X	X	X	X	X	X
MEM 3	0	1	1	X	X	X	X	X	X	X

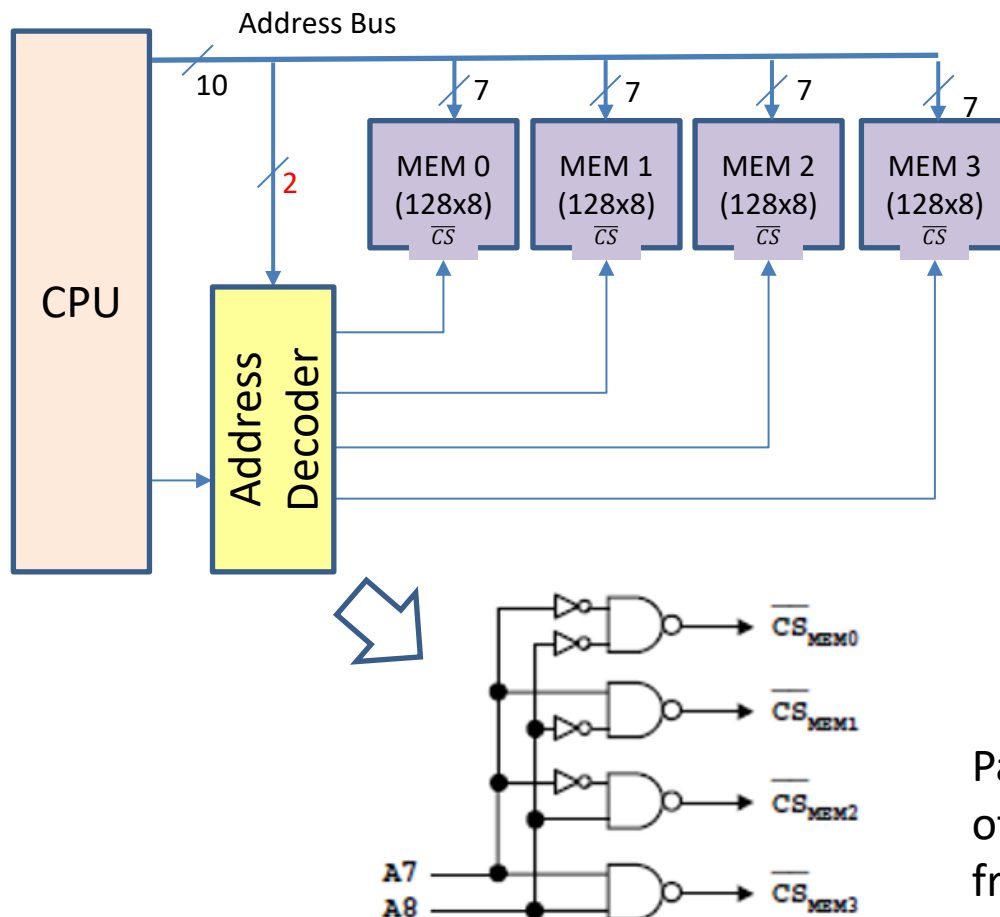


Memory Map



Partial Address Decoding

- A microprocessor has 10 address lines. Its addressable space is $2^{10} \Rightarrow$ 1KB of memory.
- Now we only want 512 bytes of memory.



	Not used			Used for Address Decoding		Used to reference memory cells on each memory IC				
Device	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0
MEM 0	X	0	0	X	X	X	X	X	X	X
MEM 1	X	0	1	X	X	X	X	X	X	X
MEM 2	X	1	0	X	X	X	X	X	X	X
MEM 3	X	1	1	X	X	X	X	X	X	X

Memory Map

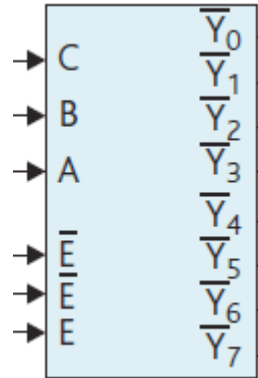
MEM 3	11 1111 1111 ₂
MEM 2	11 1000 0000 ₂
MEM 1	11 0111 1111 ₂
MEM 0	11 0000 0000 ₂
MEM 3	10 1111 1111 ₂
MEM 2	10 1000 0000 ₂
MEM 1	10 0111 1111 ₂
MEM 0	10 0000 0000 ₂
MEM 3	01 1111 1111 ₂
MEM 2	01 1000 0000 ₂
MEM 1	01 0111 1111 ₂
MEM 0	01 0000 0000 ₂
MEM 3	00 1111 1111 ₂
MEM 2	00 1000 0000 ₂
MEM 1	00 0111 1111 ₂
MEM 0	00 0000 0000 ₂

Partial address decoding prevents full use of the microprocessor's address space and frequently makes it difficult to expand the memory system at a later date

Implementation of Address Decoders

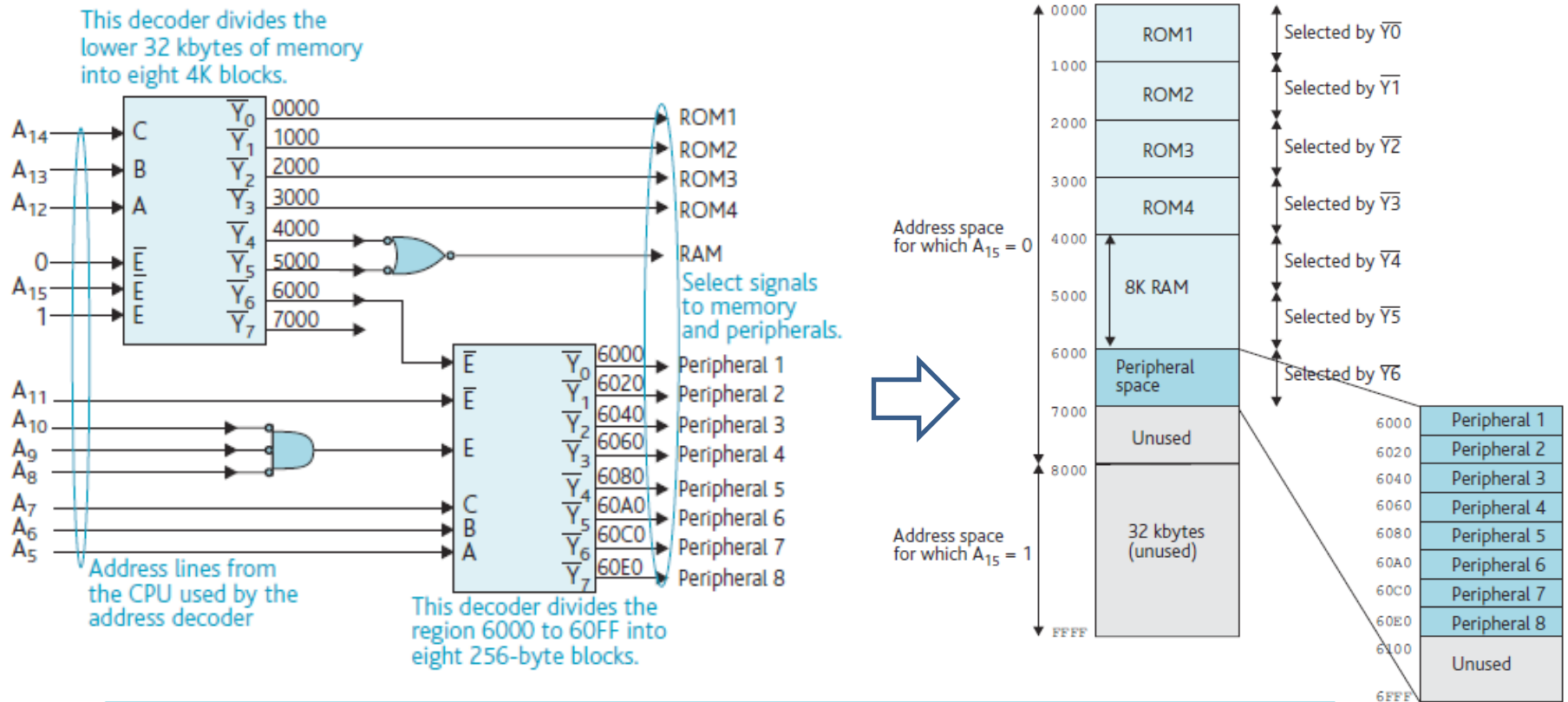
- Discrete Logic
 - High Speed
 - More difficult to design
- m-to-n Decoders
 - Selection of devices is determined by the physical wiring
 - All memory blocks must have same size
 - Can be combined with additional decoders or discrete logic to cater to memory blocks of different sizes
- Others
 - Programmable Read Only Memory (PROM)
 - Versatile, since the selection of the devices is determined by the programming
 - FPGA, PLA

Recap: 3-to-8 Decoder



Enable inputs			Control inputs			Outputs							
$\bar{E}_1$	$\bar{E}_2$	$\bar{E}_3$	C	B	A	$\bar{Y}_0$	$\bar{Y}_1$	$\bar{Y}_2$	$\bar{Y}_3$	$\bar{Y}_4$	$\bar{Y}_5$	$\bar{Y}_6$	$\bar{Y}_7$
1	1	0	X	X	X	1	1	1	1	1	1	1	1
1	1	1	X	X	X	1	1	1	1	1	1	1	1
1	0	0	X	X	X	1	1	1	1	1	1	1	1
1	0	1	X	X	X	1	1	1	1	1	1	1	1
0	1	0	X	X	X	1	1	1	1	1	1	1	1
0	1	1	X	X	X	1	1	1	1	1	1	1	1
0	0	0	X	X	X	1	1	1	1	1	1	1	1
0	0	1	0	0	0	0	1	1	1	1	1	1	1
0	0	1	0	0	1	1	0	1	1	1	1	1	1
0	0	1	0	1	0	1	1	0	1	1	1	1	1
0	0	1	0	1	1	1	1	1	0	1	1	1	1
0	0	1	1	0	0	1	1	1	1	0	1	1	1
0	0	1	1	0	1	1	1	1	1	1	0	1	1
0	0	1	1	1	0	1	1	1	1	1	1	0	1
0	0	1	1	1	1	1	1	1	1	1	1	1	0

Combining m-to-n Decoder and Discrete logic



Device	Size	Address Range	A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
ROM1	4K	0000-0FFF	0	0	0	0	x	x	x	x	x	x	x	x	x	x	x	x
ROM2	4K	1000-1FFF	0	0	0	1	x	x	x	x	x	x	x	x	x	x	x	x
ROM3	4K	2000-2FFF	0	0	1	0	x	x	x	x	x	x	x	x	x	x	x	x
ROM4	4K	3000-3FFF	0	0	1	1	x	x	x	x	x	x	x	x	x	x	x	x
RAM	8K	4000-5FFF	0	1	0	x	x	x	x	x	x	x	x	x	x	x	x	x
P1	32	6000-601F	0	1	1	0	0	0	0	0	0	0	0	x	x	x	x	x
P2	32	6020-603F	0	1	1	0	0	0	0	0	0	0	1	x	x	x	x	x
.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
P8	32	60E0-60FF	0	1	1	0	0	0	0	0	1	1	1	x	x	x	x	x